

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
ТОМСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ СИСТЕМ
УПРАВЛЕНИЯ И РАДИОЭЛЕКТРОНИКИ (ТУСУР)
Кафедра экономической математики, информатики и статистики (ЭМИС)

КОМПЬЮТЕРНАЯ ИГРА «БАЛДА»
Курсовая работа по дисциплине «Объектно-ориентированное
программирование»

Студент гр. 594-2
_____ А.А. Танцеров
«____» _____ 2026 г.

Руководитель
доцент каф. ЭМИС, к.т.н.
_____ Я.В. Костелей
«____» _____ 2026 г.

Томск 2026

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования

ТОМСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ СИСТЕМ
УПРАВЛЕНИЯ И РАДИОЭЛЕКТРОНИКИ (ТУСУР)

Кафедра экономической математики, информатики и статистики (ЭМИС)

УТВЕРЖДАЮ:
зав. кафедрой ЭМИС
д.ф.-м.н., профессор
_____ И.Г. Боровской
«___» _____ 2025 г.

ЗАДАНИЕ

На курсовую работу по дисциплине		«Объектно-ориентированное программирование»	
Студенту	Танцереву Андрею Алексеевичу		
Группа	594-2	Факультет	вычислительных систем
1. Тема	Компьютерная игра «Балда»		
2. Дата сдачи студентом курсовой работы:			
3. Исходные данные к заданию:			
3.1 Определение классов [Электронный ресурс]: metanit. URL: https://metanit.com/cpp/tutorial/5.1.php (дата обращения: 10.09.2025).			
3.2 Конструкторы (C++) [Электронный ресурс]: Microsoft Learn, 2025. URL: https://learn.microsoft.com/ru-ru/cpp/cpp/constructors-cpp?view=msvc-170 (дата обращения: 15.09.2025)			
3.3 Деструкторы (C++) [Электронный ресурс]: Microsoft Learn, 2025. URL: https://learn.microsoft.com/ru-ru/cpp/cpp/destructors-cpp?view=msvc-170 (дата обращения 15.09.2025)			
4. Структура пояснительной записки:			
4.1 Развёрнутое описание задачи			
4.2 Обзор языка программирования C++			
4.3 Выбор среды разработки			
4.4 Разработка приложения			
5. Дата выдачи задания:		2025 г.	

Руководитель: доцент каф. ЭМИС, к.т.н. _____ / Я.В. Костелей

Задание принял к исполнению: «___» ____ 2025 г. _____ / А.А Танцеров

Оглавление

Введение.....	4
1 РАЗВЁРНУТОЕ ОПИСАНИЕ ЗАДАЧИ	6
1.1 Описание предметной области	6
1.2 Пользовательские сценарии	7
1.3 Требования к приложению.....	9
1.4 Проектирование графического интерфейса	11
2 ОБЗОР ЯЗЫКА ПРОГРАММИРОВАНИЯ C++	14
2.1 От процедурного программирования к объектно-ориентированному	14
2.2 Специальные члены класса – конструктор и деструктор	15
2.3 Перечисление. Ссылка и модификатор const	16
2.4 Исключения. Обработка исключений	17
2.5 Перегрузка методов, функций, операторов. Параметры по умолчанию.	18
2.6 Наследование	19
2.7 Полиморфизм	20
2.8 Шаблоны. Интерфейсы.....	21
3 ВЫБОР СРЕДЫ РАЗРАБОТКИ	23
4 РАЗРАБОТКА ПРИЛОЖЕНИЯ	25
4.1 Описание входных и выходных данных.....	25
4.2 Описание алгоритмов	27
4.3 Описание классов и их структуры.....	28
4.4 Тестирование	32
Заключение	39
Список используемых источников.....	41
Приложение А	42

Введение

Современные системы программирования играют ключевую роль в решении практических задач автоматизации различных процессов. Язык программирования C++ остаётся одним из наиболее востребованных инструментов разработки благодаря своей производительности, гибкости и широким возможностям объектно-ориентированного программирования. Овладение данным языком и современными фреймворками для создания графических приложений является важной компетенцией для специалиста в области информационных технологий.

Целью курсовой работы является разработка компьютерного приложения «Балда» на языке программирования C++ закрепления полученных теоретических знаний и практических навыков дисциплины «объектно-ориентированное программирование».

Для достижения поставленной цели необходимо решить следующие задачи:

1. Изучить особенности предметной области, связанной с реализацией игрового приложения «Балда», и сформулировать требования к разрабатываемому программному продукту.
2. Провести обзор языка программирования C++ и выбрать инструменты разработки, подходящие для реализации приложения.
3. Выбрать графический фреймворк и другие необходимые библиотеки для создания пользовательского интерфейса.
4. Спроектировать графический интерфейс приложения и описать алгоритмы обработки игровых данных.
5. Разработать структуру классов приложения и реализовать их взаимодействие.
6. Реализовать программный код приложения с применением принципов объектно-ориентированного программирования.
7. Провести тестирование разработанного программного продукта.

В результате выполнения курсовой работы формируются следующие компетенции: понимание принципов работы современных информационных технологий и программных средств и использование их при решении задач профессиональной деятельности; разработка алгоритмов и программ, пригодных для практического применения.

Разработанное приложение будет представлять собой компьютерную реализацию классической словесной игры «Балда» с графическим интерфейсом пользователя, обеспечивающим удобное взаимодействие в режиме игры против компьютерного оппонента.

1 РАЗВЁРНУТОЕ ОПИСАНИЕ ЗАДАЧИ

1.1 Описание предметной области

Игра «Балда» представляет собой словесную логическую игру, в которой участники поочерёдно составляют слова из букв, размещённых на игровом поле. Основной задачей является формирование максимального количества существительных в именительном падеже единственного числа путём добавления по одной букве за ход и использования уже имеющихся на поле букв.

Игровое поле имеет размер 5×5 клеток. В начале игры в центральной строке поля автоматически размещается начальное слово «Балда», состоящее из пяти букв. Данное слово служит отправной точкой для формирования последующих слов и визуально выделяется жирным начертанием для отличия от букв, добавляемых в процессе игры.

Правила составления слов основаны на принципе смежности букв. Каждое новое слово должно включать в себя одну вновь добавленную букву и любое количество уже размещённых на поле букв, при условии, что все буквы слова находятся в смежных клетках по горизонтали или вертикали. Диагональная смежность не допускается. Добавляемая буква должна располагаться в свободной клетке, смежной хотя бы с одной уже занятой клеткой.

Проверка существования составленного слова осуществляется автоматически по встроенному словарю русского языка, содержащему около 14 тысяч слов. Принимаются только существительные в именительном падеже единственного числа. Каждое успешно составленное слово приносит игроку очки, количество которых равно числу букв в слове.

Целевой аудиторией приложения являются любители словесных игр, семейные пары, компании друзей и знакомых, учащиеся образовательных учреждений, филологи, а также любители кроссвордов и других интеллектуальных развлечений. Игра способствует развитию креативного

мышления, тренировке скорости реакции и принятия решений, расширению словарного запаса и улучшению знания русского языка.

Предметом автоматизации является управление игровым процессом, включающее контроль последовательности ходов пользователя и компьютерного оппонента, автоматическую проверку существования составленных слов по словарю, подсчёт набранных очков, а также ограничение времени на выполнение хода.

1.2 Пользовательские сценарии

Типичный сценарий взаимодействия пользователя с приложением включает следующую последовательность действий.

1. Запуск игровой сессии. Пользователь запускает приложение, после чего на экране отображается игровое поле размером 5×5 клеток с панелью управления, алфавитом для выбора букв и окнами статистики. Нажатие кнопки «Start» инициирует начало игры: в центральной строке поля автоматически размещается начальное слово «Балда», запускается таймер обратного отсчёта продолжительностью две минуты на выполнение хода пользователя.

2. Выбор буквы для добавления. Пользователь выбирает одну букву из представленного алфавита в нижней части окна приложения. После выбора буква становится активной для размещения на игровом поле.

3. Размещение буквы на поле. Пользователь указывает свободную клетку на игровом поле, куда должна быть добавлена выбранная буква. Клетка должна быть смежной хотя бы с одной уже занятой клеткой.

4. Составление слова. Пользователь последовательно выбирает клетки на игровом поле, формируя цепочку смежных букв, которая образует существительное в именительном падеже единственного числа. Выбранные буквы визуально выделяются красным цветом для наглядности составленной последовательности.

5. Подтверждение слова. После того как слово составлено, пользователь нажимает кнопку «Apply». Приложение проверяет составленное слово по встроенному словарю. Если слово существует и соответствует игровым правилам, добавленная буква фиксируется на поле с жирным начертанием, игроку начисляются очки в количестве, равном числу букв в слове, а составленное слово добавляется в список слов пользователя. Ход переходит к компьютерному оппоненту.

6. Обработка некорректного слова. В случае если слово не прошло проверку по словарю или не соответствует правилам составления, на экране отображается предупреждающее сообщение с указанием причины отклонения, например: «Слово XXX не найдено». Пользователь должен составить другое слово в рамках оставшегося времени на ход.

7. Ход компьютерного оппонента. После успешного подтверждения слова пользователя или истечения времени на его ход компьютер автоматически выполняет свой ход. Компьютер добавляет на поле одну случайную букву в свободную смежную клетку и составляет слово из двух букв: добавленной буквы и одной смежной с ней буквы, уже находящейся на поле. Составленное компьютером слово добавляется в список слов компьютера, ему начисляются соответствующие очки.

8. Управление игровым процессом. В любой момент игры пользователь может нажать кнопку «Pause» для приостановки игры. При этом таймер останавливается, и все действия на игровом поле блокируются до повторного нажатия кнопки для возобновления игры.

9. Просмотр статистики. На протяжении всей игры в правой части окна приложения отображается текущая статистика: список составленных слов пользователя и компьютера с указанием набранных за каждое слово очков. Таймер показывает оставшееся время на выполнение текущего хода.

10. Завершение игры. Игра завершается в момент, когда пользователь закрывает приложение. Статистика текущей игровой сессии не сохраняется между запусками приложения.

1.3 Требования к приложению

Игровое поле представлено массивом размером 5 на 5. Каждый игрок поочерёдно добавляет свою букву. Игра заканчивается, если не остаётся свободного места или невозможно составить существующее слово. Пользователь имеет возможность управлять игровым процессом следующим образом. Запуск новой игровой сессии осуществляется нажатием кнопки Start. Игру можно приостановить в любой момент нажатием кнопки Pause и возобновить повторным нажатием этой же кнопки. Составленное слово подтверждается нажатием кнопки Apply. Приложение должно автоматически проверять существование составленного слова в словаре русского языка после нажатия кнопки Apply. Если слово существует и соответствует правилам, оно засчитывается, и игроку начисляются очки. Если слово не прошло проверку, пользователь должен быть уведомлён об этом и получить возможность составить другое слово. Пользователь не может изменять или удалять буквы, размещённые на игровом поле. После добавления буквы и подтверждения слова её положение фиксируется. Приложение не должно сохранять данные об игровых сессиях между запусками. Статистика текущей игры автоматически удаляется при перезапуске приложения.

Приложение реализовано исключительно для одиночного использования. Режим игры с несколькими игроками, передающими ход друг другу, не предусматривается. Пользователь не может изменять размер игрового поля, добавлять собственные слова в словарь или модифицировать игровые правила. Приложение не предусматривает возможность сохранения игры в точках сохранения и последующей загрузки сохранённого состояния. Механизм изменения уровня сложности не реализуется – игровой процесс протекает в едином режиме для всех игровых сессий.

Приложение оперирует следующими сущностями: игровое поле – двумерный массив размером 5 на 5, содержащий буквы или пустые клетки; слово – последовательность смежных букв на игровом поле, образующая

существительное в именительном падеже единственного числа; счёт – целочисленное значение, отражающее количество набранных очков; таймер – временной интервал, отсчитывающий оставшееся время на выполнение хода; словарь – список допустимых слов русского языка.

Приложение должно функционировать на компьютере со следующими минимальными характеристиками:

- процессор Intel Core i3 или аналогичный с тактовой частотой не менее 2,0 ГГц;
- оперативная память объёмом не менее 4 ГБ; свободное место на диске не менее 100 МБ;
- наличие мыши в качестве устройств ввода;
- монитор с разрешением не менее 1280×720 пикселей.

Операционная система Windows 10 и выше. Поддержка других операционных систем, таких как Linux или macOS, не предусматривается. Для функционирования приложения не требуется установка дополнительного программного обеспечения. Все необходимые библиотеки должны быть включены в состав исполняемого файла или поставляться вместе с приложением.

Приложение не содержит конфиденциальных данных пользователей и не требует авторизации. Дополнительные требования по обеспечению безопасности, шифрованию данных или резервному копированию не предусматриваются.

Приложение должно быть разработано исключительно на языке программирования C++. Для реализации графического интерфейса использован framework – Qt.

Начальное слово в центральной строке игрового поля должно генерироваться случайным образом из predetermined набора слов при каждом запуске новой игры. Приложение не содержит уровней – игра протекает в едином непрерывном режиме. Приложение не предусматривает изменение уровня сложности. Единственный режим игры одинаков для всех

игровых сессий. Единственным параметром игрового успеха является количество набранных очков. Каждое выбранное слово приносит определенное количество очков, зависящее от количества букв в слове, одна буква – одно очко. Дополнительные параметры, такие как время выполнения, количество ходов или бонусы, не учитываются. Приложение не реализует механизм сохранения игры в точках сохранения. Игра существует только в рамках текущей сессии и не может быть возобновлена после завершения.

Тестирование приложения будет проведено методом интуитивного тестирования. Формальные тестовые сценарии не разрабатываются. Тестирование предполагает проверку работоспособности всех функциональных требований путём последовательного взаимодействия с интерфейсом приложения.

Тестирование будет проведено на компьютере со следующими характеристиками:

- процессор Intel Core i5-10300H;
- видеокарта NVIDIA GeForce GTX 1650;
- оперативная память 16 ГБ;
- операционная система Windows 11 (64-bit);
- разрешение экрана 1920×1080 пикселей;
- устройства ввода – стандартная клавиатура и мышь.

1.4 Проектирование графического интерфейса

Графический интерфейс приложения спроектирован с учётом обеспечения удобства взаимодействия пользователя с игровым процессом и наглядности представления игровых данных. Интерфейс реализован в полноэкранном режиме с фиксированным разрешением 1920×1080 пикселей.

Окно приложения включает следующие основные элементы:

- игровое поле – центральный элемент интерфейса, представляющий собой таблицу размером 5×5 клеток. Каждая клетка может содержать одну букву или оставаться пустой. Буквы начального слова «Балда» и все

добавленные в процессе игры буквы отображаются жирным начертанием. При выборе букв для составления слова они визуально выделяются красным цветом.

- панель выбора букв расположена в нижней части окна и содержит полный алфавит русского языка. Пользователь выбирает одну букву из данного набора для последующего размещения на игровом поле;

- окна статистики расположены в правой части окна приложения и разделены на два отдельных текстовых поля;

- окно слов пользователя отображает список слов, составленных пользователем, с указанием количества очков, полученных за каждое слово;

- окно слов компьютера отображает список слов, составленных компьютерным оппонентом, с указанием количества очков;

- таймер отображает оставшееся время на выполнение текущего хода пользователя в формате обратного отсчёта (минуты:секунды). Таймер останавливается при нажатии кнопки «Pause» и возобновляется при повторном нажатии;

- кнопка «Start» – запускает новую игровую сессию с размещением начального слова «Балда» в центральной строке игрового поля и активацией таймера;

- кнопка «Pause» – приостанавливает игровой процесс, блокируя все действия на игровом поле и останавливая таймер. Повторное нажатие возобновляет игру;

- кнопка «Apply» – подтверждает составленное пользователем слово и инициирует его проверку по словарю.

Запуск игры происходит при нажатии кнопки «Start» на игровом поле в центральной строке размещается слово «Балда», активируется таймер обратного отсчёта продолжительностью две минуты, и пользователь получает возможность выполнить свой первый ход. Пользователь выбирает одну букву из панели алфавита, затем указывает мышью свободную клетку на игровом поле для размещения выбранной буквы.

Пользователь последовательно выбирает клетки на игровом поле, формируя слово. Выбранные буквы выделяются красным цветом, что позволяет визуально контролировать составляемую последовательность. После составления слова пользователь нажимает кнопку «Apply». Приложение проверяет слово по словарю. Если слово корректно, оно добавляется в окно слов пользователя с указанием набранных очков, буква фиксируется на поле жирным начертанием, и ход переходит к компьютеру. Если слово некорректно, на экране отображается предупреждающее сообщение, и пользователь должен составить другое слово. При нажатии кнопки «Pause» таймер останавливается, и все действия на игровом поле блокируются. Повторное нажатие кнопки возобновляет игру.

2 ОБЗОР ЯЗЫКА ПРОГРАММИРОВАНИЯ C++

2.1 От процедурного программирования к объектно-ориентированному

C++ – это компилируемый, статически типизированный язык программирования общего назначения, разработанный как расширение языка C, добавляющий поддержку объектно-ориентированного и обобщённого программирования.

Исследуя объектно-ориентированное программирование, необходимо перейти к терминам, играющим важную роль в программировании.

Структура позволяет пользователю объединить описание некоторых полей в едином типе данных. Поле – переменная, определенная в структуре.

Структура по своему определению является пользовательским типом данных, следственно пользователь должен определить структуру (листинг 2.1). Пользователь также имеет возможность создать динамические объекты типа структуры через оператор new.

```
Struct <Имя структуры> {  
    <тип_поля_1> <имя_поля_1> [ =<инициализационное_значение>];  
    <тип_поля_2> <имя_поля_2> [ =<инициализационное_значение>];  
    ...  
} [ <имя_переменной_типа_структуры_1> [, ...]];
```

Листинг 2.1 – Пример структуры

Переходя к более сложным типам данных, необходимо узнать о классах. Класс является сложным типом данных, который хранит в себе данные и их алгоритмы изменения. Класс описывает структуру и действия объектов. Также класс состоит из полей и методов. Поле – переменная или объект, объявленный в классе, хранящая данные конкретного объекта, а метод – блок кода, выполняющий определенное действие или задачу [1].

Основной принцип, реализуемый классами, является инкапсуляция. Инкапсуляция – объединение данных и методов, которые их обрабатывают, в одно целое, скрывание внутренних деталей от пользовательского кода.

Скрытие реализации обеспечивается благодаря модификаторам:

1. Модификатор public: члены доступны самому классу и его потомкам.

2. Модификатор `private`: члены класса, доступные только методам самого этого класса.

3. Модификатор `protected`: члены класса, доступные самому классу и его потомкам, актуально для наследования.

2.2 Специальные члены класса – конструктор и деструктор

Удобно, когда при создании объекта, передаются данные, которые устанавливают начальное состояние объекта. Этот механизм называется конструктором.

Конструктор – это метод, который вызывается самостоятельно, без помощи пользователя, при инициализации нового объекта [2]. Пример оформления конструктора представлен на листинге 2.2.

```
<имя_класса> ( [<список_формальных_параметров>] ) {  
    <тело_конструктора> // присваивание начальных значений полям  
}
```

Листинг 2.2 – Оформление конструктора

Наименование конструктора такое же, как и имя класса. Инициализация полей должна происходить в теле конструктора.

Без воздействия пользователя конструктор вызывается:

- со статическим объектом;
- при захвате динамической памяти;
- для каждого элемента массива;

Компилятор создаёт пустой конструктор по умолчанию, то есть без формальных параметров, если пользователь не создаст свой.

Деструктором называют метод класса, предназначенный для уничтожения экземпляра класса, вызывается не пользователем, так как могут возникнуть ошибки [3]. Наименование деструктора такое же, как и у класса, однако перед этим ставится знак тильды – «~». Пример оформления деструктора представлен на листинге 2.3.

```

~ <имя_класса> {
    <тело_деструктора>
}

```

Листинг 2.3 – Определение деструктора

Деструктор вызывается:

- после выполнения оператора delete для динамических объектов;
- перед завершением приложения для глобальных и статических объектов;
- после выхода из блока видимости для локальных объектов [3].

Статическое поле – общая переменная, с модификатором static, для всех объектов, а статический метод – метод, вызываемый для класса, с модификатором static. Инициализация статического поля внутри класса невозможна. Доступ к статическим полям и методам происходит через оператор пространства имён «::» по имени класса.

2.3 Перечисление. Ссылка и модификатор const

Перечисление (enum) – набор целочисленных констант, которые определяются пользователем, пример представлен на листинге 2.4. Изначально первому элементу присваивается значение нуля, следующему единицы и так далее, однако можно задать значение константы через присваивание. А для ограничения области видимости элементов и недопущения преобразований в целочисленные типы используется типобезопасное перечисление – enum class.

```

enum Color {
    Red,
    Blue = 10,
    Green,
    LightRed = 0
};

void main {
    int colorInt = 1;
    Color colorEnum = Red;
    colorInt = colorEnum;
}

```

Листинг 2.4 – Пример перечисления

Ссылка – альтернативный способ задания имени переменной, благодаря которой к ней можно обратиться. При объявлении используется символ амперсанта – «&». Передача может осуществляться по значению, адресу или ссылке.

Для передачи данных ссылке используют, чтобы:

- изменять исходный объект в функции;
- экономить ресурсы, не используя копирования.

Ключевое слово `const` является способом запрета изменения объектов или данных [4].

Константой является значение, которое становится известно до запуска приложения, когда константная переменная имеет значение, которое рассчитывается во время выполнения программы.

2.4 Исключения. Обработка исключений

Ошибки выполнения – ошибки, которые связаны с пользовательским вводом данных, недопустимыми ресурсами, возникающие при выполнении работы. В C++ существуют конструкции для обработки исключений: `try-catch-throw`.

Исключение – описание исключительной ситуации, являющийся ошибкой или альтернативным исходом событий. Исключение предполагает собой, что функция или метод произошёл и получил результат [5]. Целью исключения является описание проблемы для дальнейшего объявления о ней и для её решения. Далее исключение можно бросить, то есть применяется оператор, сигнализирующий о исключительной ситуации. Пример реализации исключений представлен на листинге 2.5.

```
try {  
    if (b == 0) throw std::runtime_error("Division by zero!");  
    result = a / b;  
} catch (const std::runtime_error& e) {  
    std::cout << "Error: " << e.what() << std::endl;  
}
```

Листинг 2.5 – Пример реализации исключений

Целью обработки исключения является не допущение попадания в операционную систему, в противном случае повлечет вылет программы.

Бросить и словить в C++ можно:

- значения простых данных;
- строки;
- объект класса;
- любой объект.

При операторе `catch`, заданном трёхточием (обработчик по умолчанию), блок обработает каждый тип исключения, если его не поймал `catch` с конструкции.

2.5 Перегрузка методов, функций, операторов. Параметры по умолчанию

Перегрузкой называют механизм, благодаря которому для одного имени можно определить несколько функций, различающихся списком параметров.

Оператор в синтаксисе языка – символ или сочетание символов, используемые для выполнения определенной операции. Перегрузкой оператора называют механизм, который позволяет использовать символы операторов вместо имени функции или методов с целью внесения логики обработки объектов. Задается ключевым словом `operator` и символом оператора. Шаблон реализации представлен на листинге 2.6.

```
<тип возвращаемого значения> operator <символ оператора> (  
    [<тип> <первый_операнд> ,  
    [<тип> <второй_операнд> ] ] )  
{  
    <тело оператора как у функции или метода>  
    return <возвращаемое значение>;  
}
```

Листинг 2.6 – Шаблон реализации перегрузки оператора

Параметры по умолчанию – механизм определения функции или метода, при котором они принимают аргумент по умолчанию, то есть отсутствие при вызове аргумента, применяется параметр, заданный в самом методе [6]. Пример оформления параметров по умолчанию представлен на листинге 2.7.

```

<тип_возвращаемого_значения> имя_функции (
[ <тип_1> обязательный_параметр_1
[, [ <тип_2> обязательный_параметр_2
[, ...] ] ]
[, <тип_3> необязательный_параметр_1 = <значение_по_умолчанию_1>
[, [ <тип_2> необязательный_параметр_2 = <значение_по_умолчанию_2>
[, ...] ] ]
) ...

```

Листинг 2.7 – Оформление параметров по умолчанию

2.6 Наследование

Наследованием называют механизм определения класса, при котором класс включаются поля и методы не только свои, но и с другого класса. Базовым классом называют такой, от которого наследуются, а который наследует – уже производным. Шаблон реализации наследования представлен на листинге 2.8. Модификатором наследования называют правила, будут ли видимы внутри произвольного класса, наследуемые члены [7].

```

class <Имя_ребенка> : <модификатор_наследования> <Имя_родителя> {
... // обычное определение членов класса
};

```

Листинг 2.8 – Шаблон реализации наследования

Модификаторы доступа различают:

- public – член класса видно за его пределами;
- private – поле или метод видны только в классе;
- protected – поле или метод видны только в классе или в его потомке.

Конструкторы и деструкторы, а также перегрузки операторов, явно не наследуются, но при создании и уничтожения объекта производного класса они будут вызываться.

К тому же, конструкторов может существовать несколько, причём все, кроме одного, с параметрами, для чего существует метод явного вызова конструктора базового класса при определении производного класса, где в конструктор базового класса передаются значения формальных параметров или констант. Пример шаблона явного вызова конструктора приведен на листинге 2.9.

```

<Имя_конструктора_ребенка>( [ формальные_параметры ] )
[ : <Имя_конструктора_родителя> ( [ фактические_параметры ] ) ] {
<тело_конструктора>
}

```

Листинг 2.9 – Шаблон явного вызова конструктора базового класса

2.7 Полиморфизм

Ранее вводилось понятие наследования, которое определяло реализацию перегрузки одноименных методов в классе. Однако, существует также понятие переопределения, то есть замена существующего метода или оператора в производном классе, для чего существует функция виртуального метода (virtual). Шаблон реализации переопределения представлен на листинге 2.10. В производных классах, если название и параметры совпадают с виртуальным, то он переопределяется и вносится как виртуальный.

```

virtual <тип_возвращаемого_значения> имя_метода (
[ формальные_параметры ] ) {
// тело метода
}

```

Листинг 2.10 – Шаблон переопределения

Для переопределяющих методов производных классов, чтобы повысить читаемость кода и профилактики ошибок, можно использовать ключевое слово `override` [8], указывающий, что этот метод используется для переопределения базового класса. Если, в случае не нахождения подходящего виртуального метода, компилятор вызовет ошибку. Ключевое слово `final` позволяет запретить переопределение виртуального метода в производных классах.

Также можно привести переопределение метода, не имея реализации в базовом классе, определив как чистый виртуальный код, то есть метод, реализующиеся только в производных классах. Шаблон реализации чистого виртуального кода представлен на листинге 2.11.

```

virtual имя_метода ( [ формал_парам ] ) = 0;

```

Листинг 2.11 – Шаблон чистого виртуального кода

Абстрактным классом называют класс, в котором имеется, как минимум, один чистый виртуальный метод, использующийся для наследования и указания типов ссылок и указателей для объектов производных классов. Он используется для определения общего интерфейса и структуры классов, наследующиеся от него.

Если класс содержит виртуальные функции, то его часто называют полиморфным, так как позволяет применять полиморфизм, то есть обрабатывать данные разного типа.

2.8 Шаблоны. Интерфейсы

Шаблоны – механизм создания функций, методов и классов, являющихся универсальными, способные обрабатывать разные типы данных [9]. Синтаксис определения шаблонной функции представлен на листинге 2.12. `template` определяет шаблон, в «`<>`» записывается список параметров шаблона.

```
template < typename TS [, class TC[,...]] > // список параметров шаблона
TS имя_функции (
[TS формальный_параметр_1
[, TC формальный_параметр_2
[, ...] ]]) {
// тело функции
}
```

Листинг 2.12 – Шаблон реализации шаблонной функции

Определение шаблонного класса задаётся аналогично с функцией, только ключевое слово `template` указывается перед определением класса. В шаблонах можно использовать не только параметры-типы, но и параметры-константы, также им можно задавать параметры по умолчанию.

Интерфейс – сложный тип данных, определенный на уровне синтаксиса, состоящий из чистых виртуальных функций и констант, в то же время не содержит поля и методы реализаций. Интерфейс позволяет:

- взаимозаменять модули, то есть при наличии одинакового интерфейса у разных объектов, можно заменить один объект другим;

- уменьшать число зависимостей, то есть достаточно знание интерфейса для работы с объектом;
- уменьшать число зависимостей через ограничение доступных методов, то есть интерфейсы позволяют выборочно предоставить доступ к функциям;
- множественное наследование через интерфейсы, не имеющие проблем неопределенности при множественном наследовании, так как конкретные реализации в интерфейсе не указаны.

3 ВЫБОР СРЕДЫ РАЗРАБОТКИ

Для разработки приложения и графического интерфейса игры «Балда» использована интегрированная среда разработки Qt Creator. Данный framework представляет собой кроссплатформенную среду разработки, специально предназначенную для создания приложений с использованием фреймворка Qt.

Qt Creator обеспечивает полный цикл разработки программного обеспечения, включая редактирование кода, визуальное проектирование пользовательских интерфейсов, отладку и развёртывание приложений. Среда поддерживает разработку на языках программирования C++ и QML, что делает её универсальным инструментом для создания как классических десктопных приложений, так и современных интерфейсов с декларативным описанием.

Среда предоставляет мощный текстовый редактор с подсветкой синтаксиса для языка C++, автодополнением кода, навигацией по определениям классов и функций, а также рефакторингом кода. Встроенная система подсказок упрощает работу с обширным API фреймворка Qt, предоставляя контекстно-зависимую документацию непосредственно в редакторе. Qt Creator интегрирует инструмент Qt Designer, позволяющий визуально проектировать графические интерфейсы пользователя в режиме WYSIWYG (What You See Is What You Get). Разработчик может размещать элементы управления на форме, настраивать их свойства и связывать сигналы с слотами без необходимости вручную писать код инициализации интерфейса. Это существенно ускоряет процесс создания пользовательских интерфейсов и снижает вероятность ошибок.

Среда поддерживает несколько систем сборки, включая qmake, CMake и Qbs. В рамках данного проекта использована система сборки qmake, которая является стандартным инструментом для проектов на базе Qt. Qmake автоматически генерирует файлы сборки (Makefile) на основе файла проекта с

расширением .pro, что упрощает процесс компиляции и компоновки приложения.

Выбор Qt Creator в качестве среды разработки обусловлен следующими преимуществами:

- тесная интеграция с фреймворком Qt. Qt Creator разработан компанией The Qt Company специально для работы с фреймворком Qt, что обеспечивает наилучшую совместимость, производительность и удобство использования всех возможностей фреймворка;

- визуальное проектирование интерфейсов. Инструмент Qt Designer существенно упрощает процесс создания графических интерфейсов, позволяя сосредоточиться на функциональной логике приложения, а не на рутинном написании кода инициализации элементов управления;

- бесплатность и открытый исходный код. Qt Creator распространяется под лицензией GPL v3 и является полностью бесплатным инструментом с открытым исходным кодом, что делает его доступным для образовательных целей и некоммерческой разработки;

- широкая документация и сообщество. Qt Creator имеет обширную официальную документацию, множество обучающих материалов и активное сообщество разработчиков, что облегчает процесс изучения и решения возникающих проблем;

- стабильность и надёжность. Среда разработки прошла длительный путь развития и используется в коммерческих и промышленных проектах по всему миру, что свидетельствует о её стабильности и надёжности.

Таким образом, Qt Creator представляет собой оптимальный выбор для разработки приложения «Балда», обеспечивая все необходимые инструменты для эффективной разработки, отладки и развёртывания программного продукта на языке C++ с использованием фреймворка Qt.

4 РАЗРАБОТКА ПРИЛОЖЕНИЯ

4.1 Описание входных и выходных данных

Основным источником данных является текстовый файл `word_rus.txt`, содержащий список допустимых слов русского языка. Файл встроен в исполняемый файл приложения в качестве ресурса Qt и доступен по пути «`:/dict/word_rus.txt`». Структура файла представляет собой последовательность строк, где каждая строка содержит одно слово – существительное в именительном падеже единственного числа. Общее количество слов в словаре составляет приблизительно 14 тысяч записей.

При загрузке словаря выполняется чтение файла построчно с последующим преобразованием каждого слова в верхний регистр и удалением пробельных символов. Все обработанные слова сохраняются в множестве «`QSet<QString>`», что обеспечивает быстрый поиск при проверке существования слова с временной сложностью « $O(1)$ » в среднем случае.

Пользовательские воздействия на элементы интерфейса. Приложение принимает следующие типы пользовательских действий:

- выбор буквы из алфавита. Пользователь выбирает одну букву русского алфавита из представленной в нижней части окна панели кнопок. Выбранная буква передаётся в игровое поле для последующего размещения.
- клики на клетки игрового поля. Пользователь взаимодействует с клетками игрового поля размером 5×5 путём нажатия левой кнопкой мыши. Первый клик на пустую клетку размещает выбранную букву. Последующие клики на занятые клетки добавляют буквы в составляемое слово. Повторный клик на последнюю выбранную букву отменяет её выбор.
- нажатие кнопки «Start». Иницирует запуск новой игровой сессии с автоматическим размещением начального слова «Балда» в центральной строке поля и активацией таймера обратного отсчёта.
- нажатие кнопки «Pause». Переключает состояние игры между режимами приостановки и возобновления. При активной игре останавливает

таймер и блокирует действия на поле. При приостановленной игре возобновляет отсчёт времени.

- нажатие кнопки «Apply». Подтверждает составленное пользователем слово и инициирует его проверку по словарю.

В качестве начального слова, размещаемого в центральной строке игрового поля при запуске новой игровой сессии, используется константное значение «БАЛДА». Данное слово жёстко закодировано в методе `placeStartWord` класса `BaldaBoard` и не изменяется между игровыми сессиями.

Игровое поле представлено в виде таблицы размером 5×5 клеток, реализованной с помощью класса `QGridLayout` и набора кнопок `QPushButton`. Каждая клетка может находиться в одном из трёх состояний:

- пустая клетка – не содержит текста, доступна для размещения новой буквы.
- клетка с буквой – содержит одну букву русского алфавита, отображаемую жирным шрифтом размером 18pt.
- выбранная клетка – буква отображается красным цветом, что указывает на её включение в составляемое пользователем слово.

Приложение отображает два отдельных списка слов в элементах `QListWidget`, расположенных в правой части окна:

- список слов игрока содержит все слова, успешно составленные и подтверждённые пользователем. Каждое слово отображается в виде отдельной строки.
- список слов компьютера содержит все слова, составленные компьютерным оппонентом в процессе своих ходов.

Оба списка автоматически обновляются при успешном подтверждении нового слова путём добавления записи в конец списка.

Оставшееся время на выполнение хода пользователя отображается в формате «mm:ss» («минуты:секунды») в элементе `QLabel` в верхней части окна. Значение таймера обновляется каждую секунду. При запуске хода таймер устанавливается в значение «02:00» (две минуты) и уменьшается до

«00:00». По достижении нулевого значения ход автоматически передаётся компьютерному оппоненту.

Текущая выбранная пользователем буква из алфавита отображается в элементе QLabel с текстом «Буква: [буква]». До выбора буквы отображается текст «Буква: []». При обнаружении некорректного ввода данных приложение отображает диалоговое окно класса QMessageBox с предупреждающим сообщением. Основные типы сообщений: «Слова "[слово]" нет в словаре». Отображается при попытке подтвердить несуществующее или уже использованное слово.

Диалоговое окно содержит кнопку «ОК» для подтверждения прочтения сообщения и закрытия окна.

Приложение использует следующие методы визуального выделения для повышения наглядности:

- буквы на игровом поле по умолчанию отображаются чёрным цветом жирным шрифтом размером 18 pt.
- буквы, выбранные пользователем для составления слова, временно меняют цвет на красный.
- после подтверждения слова все буквы возвращаются к стандартному оформлению (чёрный цвет, жирный шрифт).

4.2 Описание алгоритмов

В процессе разработки приложения «Балда» реализован ряд алгоритмов, обеспечивающих игровую логику, проверку данных и управление состоянием приложения. Ниже приведено описание ключевых алгоритмов с указанием их назначения и логики работы, их реализация представлена в приложении А.

Алгоритм загрузки словаря реализован в методе loadDictionary класса BaldaBoard и выполняется при инициализации игрового поля в конструкторе класса. Назначение алгоритма: чтение словаря допустимых слов из встроенного ресурсного файла и сохранение его в структуре данных для последующей быстрой проверки существования слов.

Использование структуры данных `QSet<QString>` обеспечивает эффективную проверку наличия слова в словаре за константное время в среднем случае.

Алгоритм проверки слова реализован в методе `checkWord` класса `BaldaBoard` и вызывается при подтверждении слова пользователем или при попытке компьютерного оппонента составить слово. Назначение алгоритма: определение корректности составленного слова путём проверки его наличия в словаре и отсутствия в списке уже использованных слов.

Алгоритм хода компьютерного оппонента реализован в методе `computerMove` класса `BaldaBoard` и выполняется после успешного подтверждения слова игроком или истечения времени на его ход. Назначение алгоритма: автоматическое выполнение хода компьютером путём размещения одно

Алгоритм обработки клика на игровом поле реализован в методе `onBoardClicked` класса `BaldaBoard` и вызывается при нажатии пользователем на любую клетку игрового поля. Назначение алгоритма: обработка пользовательских действий по размещению новой буквы на поле и выбору

Алгоритм работы таймера реализован в методе `timeOutSlot` класса `BaldaBoard` и вызывается автоматически каждую секунду при активном таймере. Назначение алгоритма: уменьшение оставшегося времени на ход игрока и автоматическая передача хода компьютеру при истечении времени.

4.3 Описание классов и их структуры

Приложение «Балда» разработано с использованием объектно-ориентированного подхода и включает три основных класса, каждый из которых отвечает за определённый аспект функционирования программы. Архитектура приложения построена на принципах разделения ответственности, где класс `BaldaWindow` отвечает за пользовательский интерфейс, класс `BaldaBoard` реализует игровую логику, а класс `BaldaWord` предназначен для работы со словарём.

Взаимодействие между классами осуществляется через механизм сигналов и слотов фреймворка Qt, что обеспечивает слабую связанность компонентов и упрощает сопровождение кода.

Класс BaldaWindow реализует видимость главного окна приложения, содержащего все элементы пользовательского интерфейса и обеспечивающего взаимодействие между пользователем и игровой логикой. Класс наследуется от QMainWindow – базового класса Qt для создания главных окон приложений. На листинге 4.1 объявлен класс BaldaWindow.

```
class BaldaWindow : public QMainWindow
{
    Q_OBJECT

public:
    explicit BaldaWindow(QWidget *parent = nullptr);

private slots:
    void onLetterClicked(QAbstractButton *button);
    void onStartClicked;

private:
    BaldaBoard *board;
    QPushButton *startBtn;
    QPushButton *pauseBtn;
    QPushButton *applyBtn;
    QLabel *timeLabel;
    QLabel *letterLabel;
    QListWidget *playerWords;
    QListWidget *computerWords;
    QButtonGroup *letterGroup;

    void createLetterButtons(QVBoxLayout *parentLayout);
};
```

Листинге 4.1 – Объявление класса BaldaWindow

Класс BaldaBoard реализует игровое поле и всей игровой логики, включая обработку пользовательских действий, проверку слов, выполнение хода компьютера и управление таймером. Класс наследуется от QFrame – базового класса Qt для создания виджетов-контейнеров с возможностью визуального оформления рамкой. На листинге 4.2 представлено объявление класса BaldaBoard.

Класс BaldaWord предоставляет функциональность для работы со словарём допустимых слов и списком использованных слов. Класс разработан как вспомогательный компонент для инкапсуляции логики работы со словами.

Однако, в текущей реализации приложения класс BaldaWord объявлен, но не используется. Вся функциональность работы со словарём интегрирована непосредственно в класс BaldaBoard для упрощения архитектуры. Данный класс может быть использован в будущих версиях приложения для улучшения структуры кода и разделения ответственности.

На листинге 4.3 представлено объявление класса BaldaWord.

```
class BaldaBoard : public QFrame
{
    Q_OBJECT

public:
    explicit BaldaBoard(QWidget *parent = nullptr);
    void curletter(const QString &l);

signals:
    void timeChanged(const QString &);
    void pl_addWord(const QString &);
    void comp_addWord(const QString &);

public slots:
    void start;
    void pause;
    void apply;

private slots:
    void onBoardClicked(int);
    void timeOutSlot;

private:
    void placeStartWord;
    void loadDictionary;
    bool checkWord(const QString &word);
    void computerMove;

private:
    QButtonGroup *board;
    QVector<QPushButton*> player;
    QString letter;
    QString playerWord;
    int curPos;
    const int rowSize = 5;
    const int colSize = 5;
    bool addLetter;
    bool isStarted;
    bool isPlayerTurn;
    QTimer timer;
    QTime time;
    QSet<QString> dictionary;
    QSet<QString> usedWords;
};
```

Листинг 4.2 – Объявление класса BaldaBoard

```

class BaldaWord
{
public:
    BaldaWord;
    void loadDictionary(const QSet<QString>& dict);
    bool exists(const QString& word) const;
    bool wasUsed(const QString& word) const;
    void addUsed(const QString& word);

private:
    QSet<QString> dictionary;
    QSet<QString> usedWords;
};

```

Листинг 4.3 – Объявление класса BaldaWord

Взаимодействие между объектами классов BaldaWindow и BaldaBoard осуществляется через механизм сигналов и слотов фреймворка Qt. При инициализации главного окна в конструкторе класса BaldaWindow устанавливаются следующие связи:

1. Сигнал clicked кнопки «Start» связывается со слотом onStartClicked класса BaldaWindow, который вызывает метод start объекта BaldaBoard.
2. Сигнал clicked кнопки «Pause» напрямую связывается со слотом pause объекта BaldaBoard.
3. Сигнал clicked кнопки «Apply» напрямую связывается со слотом apply объекта BaldaBoard.
4. Сигнал timeChanged объекта BaldaBoard связывается со слотом setText метки timeLabel для автоматического обновления отображаемого времени.
5. Сигнал pl_addWord объекта BaldaBoard связывается с лямбда-функцией, добавляющей слово в список playerWords.
6. Сигнал comp_addWord объекта BaldaBoard связывается с лямбда-функцией, добавляющей слово в список computerWords.
7. Сигнал buttonClicked группы кнопок букв связывается со слотом onLetterClicked класса BaldaWindow, который передаёт выбранную букву в метод curletter объекта BaldaBoard.

Такая организация взаимодействия обеспечивает слабую связанность компонентов: класс BaldaWindow не содержит игровой логики и отвечает

исключительно за пользовательский интерфейс, в то время как класс BaldaBoard полностью инкапсулирует игровую логику и не зависит от деталей реализации интерфейса.

Точка входа в приложение реализована в файле main.cpp и содержит стандартную структуру приложения Qt.

Код main.cpp представлен на листинге 4.4.

```
#include "baldawindow.h"
#include <QApplication>
#include <QTextCodec>

int main(int argc, char *argv[])
{
    QApplication a(argc, argv);

    // Установка кодировки для русских букв
    QTextCodec *cyrillicCodec = QTextCodec::codecForName("Windows-1251");
    QTextCodec::setCodecForLocale(cyrillicCodec);

    BaldaWindow w;
    w.show;

    return a.exec;
}
```

Листинг 4.4 – Код реализации main.cpp

Функция main выполняет следующие действия:

1. Создаёт объект приложения QApplication, необходимый для функционирования любого приложения Qt с графическим интерфейсом.
2. Устанавливает кодировку локали в Windows-1251 для корректной работы с русскими символами.
3. Создаёт объект главного окна BaldaWindow.
4. Отображает главное окно методом show.
5. Запускает главный цикл обработки событий приложения методом exec.

4.4 Тестирование

Тестирование приложения «Балда» проведено методом интуитивного тестирования, которое предполагает проверку работоспособности

функциональных требований путём последовательного взаимодействия с интерфейсом приложения без разработки формальных тестовых сценариев.

Тестирование проводилось путём последовательной проверки работоспособности основных функций приложения.

При запуске приложения и нажатии кнопки «Start» на игровом поле в центральной строке автоматически размещается начальное слово «БАЛДА», таймер устанавливается в значение «02:00» и начинает обратный отсчёт. Игровое поле активируется для взаимодействия. Результат представлен на рисунке 4.1.

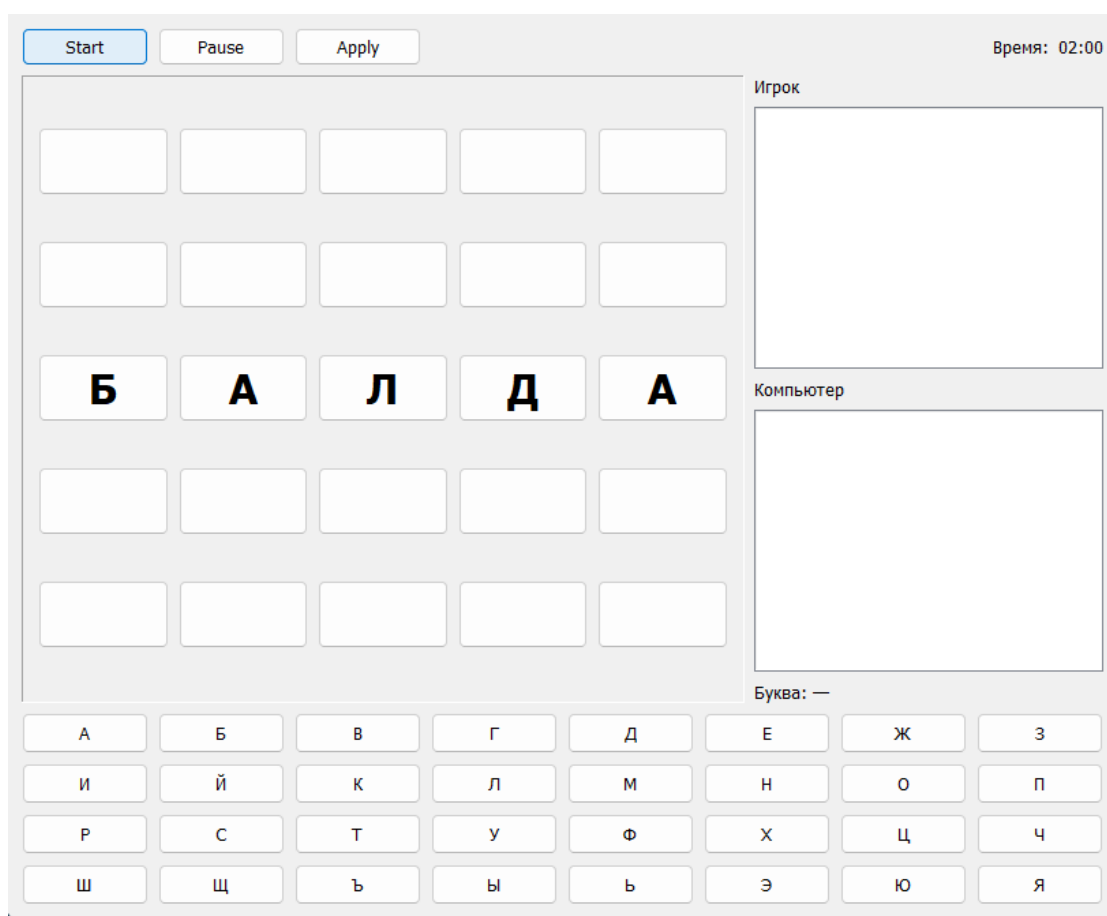


Рисунок 4.1 – Скриншот начала игры

Проверка приостановки игры показала, что при нажатии кнопки «Pause» таймер останавливается, все действия на игровом поле блокируются. Повторное нажатие кнопки возобновляет отсчёт времени и восстанавливает возможность взаимодействия. Результат представлен на рисунке 4.2.

Выбор буквы из алфавита осуществляется нажатием соответствующей кнопки в нижней панели. Выбранная буква отображается в метке «Буква», кнопка визуально изменяет состояние. Результат представлен на рисунке 4.3.

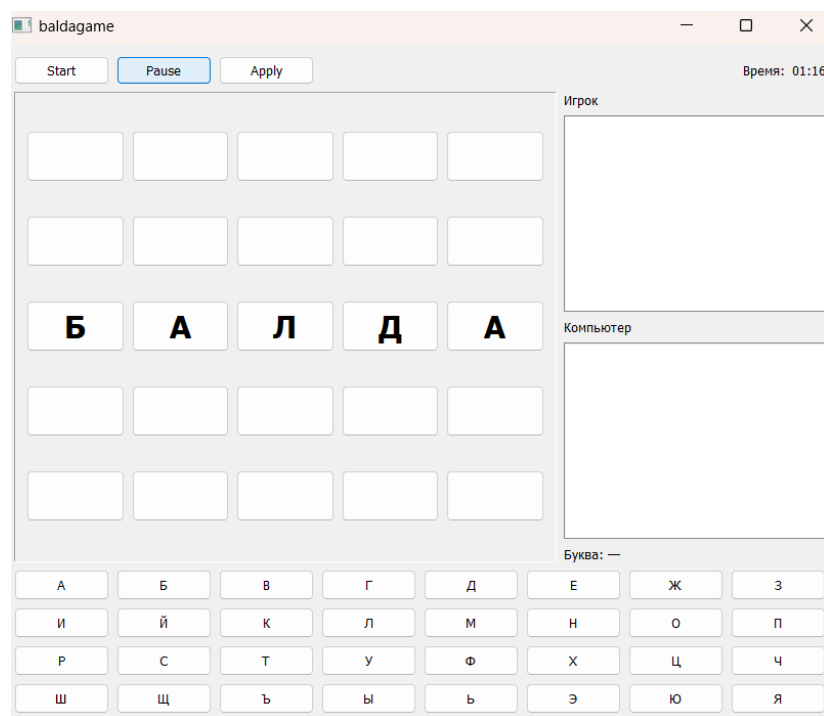


Рисунок 4.2 – Скриншот остановки игры

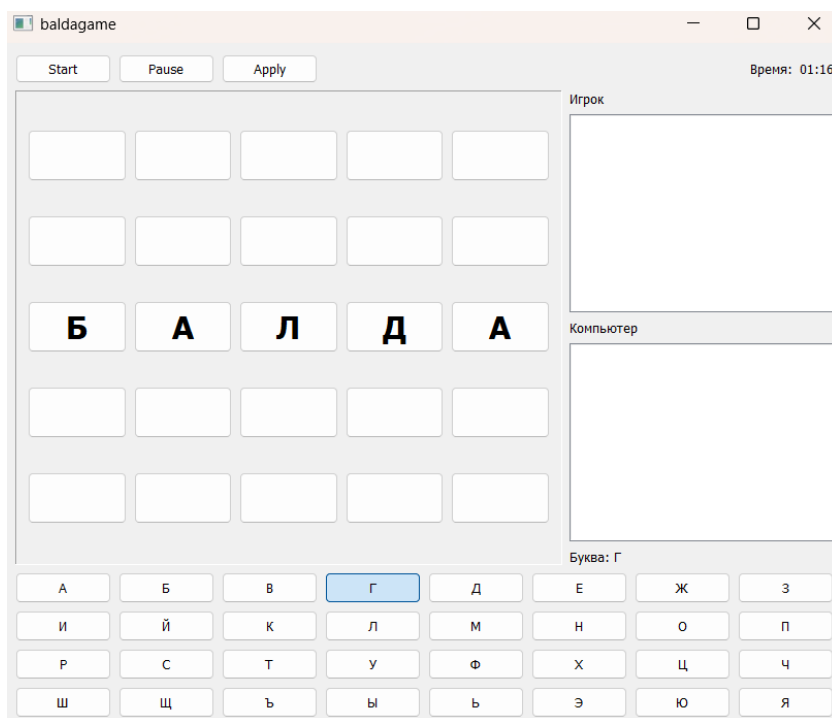


Рисунок 4.3 – Скриншот с выбранной буквой

Размещение выбранной буквы на игровом поле выполняется кликом на пустую клетку. Буква появляется в выбранной клетке и отображается красным цветом. Результат представлен на рисунке 4.4.

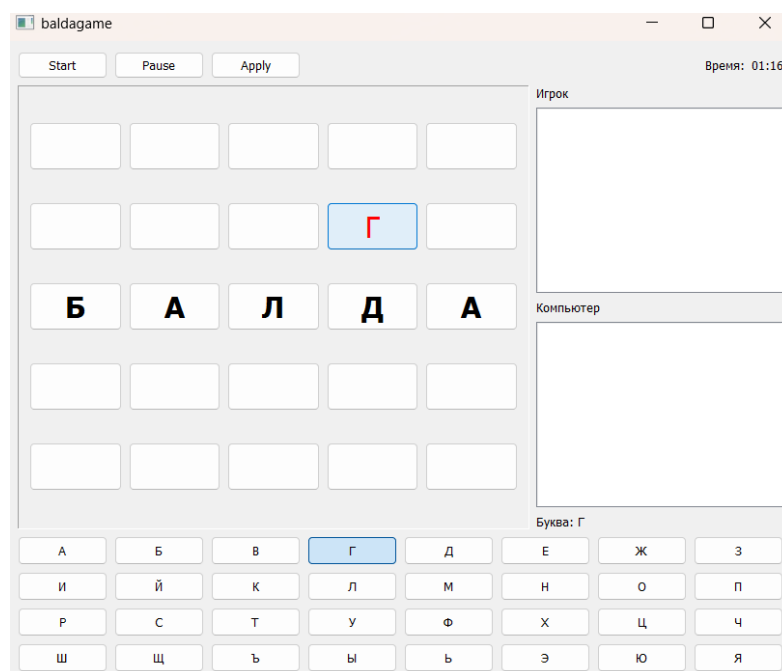


Рисунок 4.4 – Скриншот вставки буквы на поле

Составление слова происходит путём последовательного выбора клеток с буквами на поле. Выбранные буквы визуально выделяются красным цветом, формируя составляемое слово. Результат представлен на рисунке 4.5.

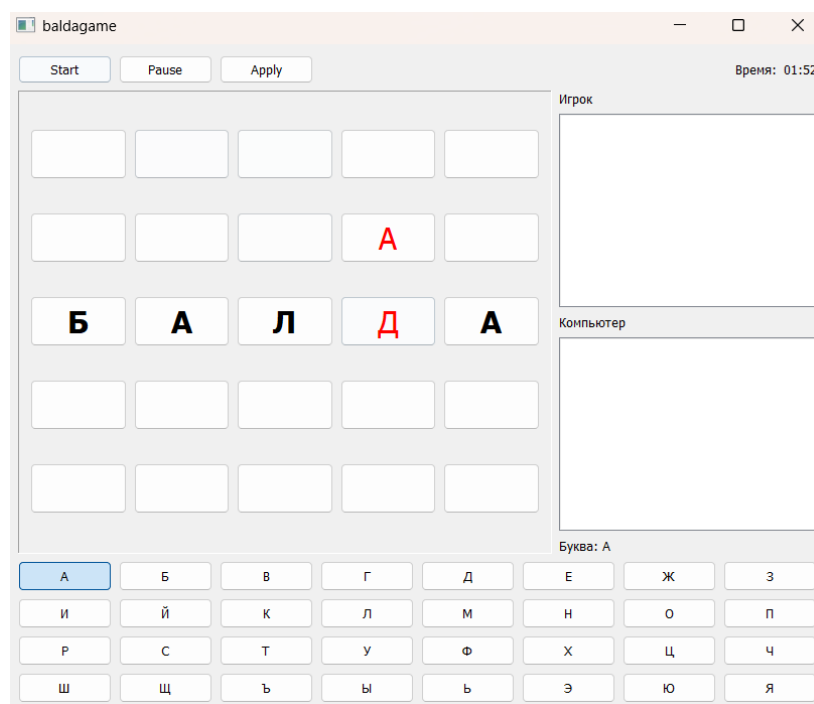


Рисунок 4.5 – Выделение выбранного слова

При подтверждении корректного слова кнопкой «Apply» слово добавляется в список слов игрока, визуальное выделение букв сбрасывается, ход автоматически переходит к компьютеру. В процессе игры в правой панели интерфейса отображаются списки составленных слов игрока и компьютера. Результат представлен на рисунке 4.6.

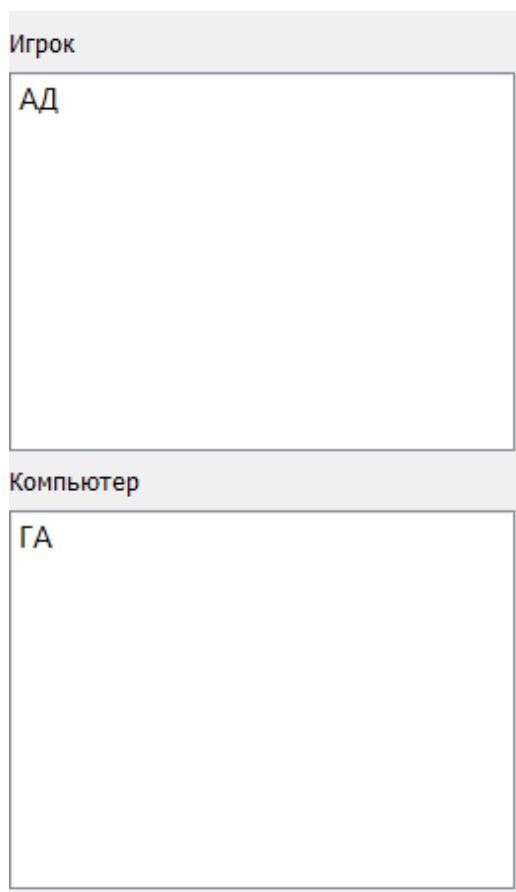


Рисунок 4.6 – Скриншот добавления слова в список

При попытке подтверждения несуществующего или уже использованного слова отображается диалоговое окно с сообщением об ошибке. Слово не засчитывается, ход остаётся за игроком. Результат представлен на рисунке 4.7.

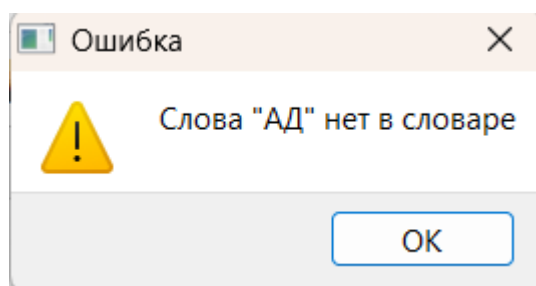


Рисунок 4.7 – Скриншот об ошибке

При истечении времени на ход (достижение таймером значения «00:00») ход автоматически переходит к компьютерному оппоненту без выполнения действий игроком. Результат представлен на рисунке 4.8.

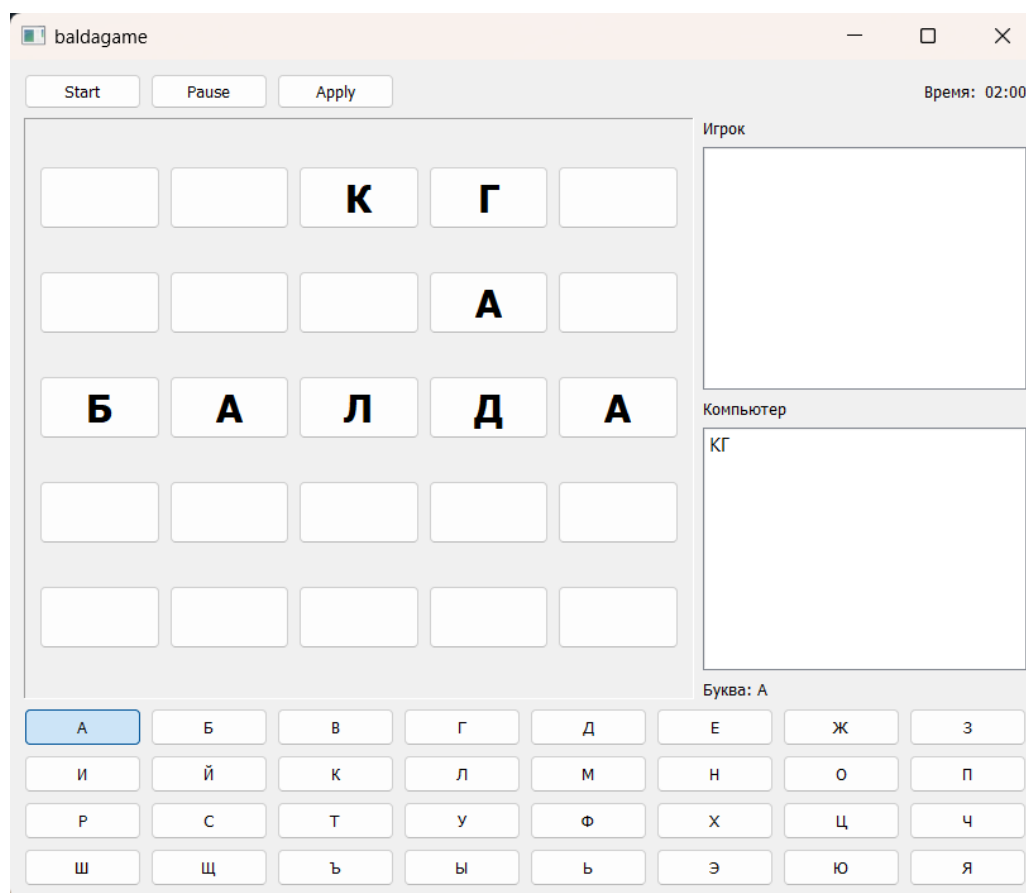


Рисунок 4.8 – Скриншот истечения времени игрока

Проверка ограничений показала, что повторное использование ранее составленного слова не допускается – приложение выводит сообщение об ошибке. Буквы, размещённые и подтверждённые на поле, не могут быть изменены или удалены.

После закрытия и повторного запуска приложения игровое поле пусто, списки слов очищены, статистика предыдущей игры отсутствует, что подтверждает отсутствие сохранения данных между сессиями.

Проведённое интуитивное тестирование подтвердило работоспособность всех основных функций приложения «Балда». Проверенные функции выполняются корректно, пользовательский интерфейс отзывчив и удобен в использовании.

Приложение стабильно работает на тестовой машине с характеристиками, превышающими минимальные требования. Производительность приложения высокая, задержки при обработке пользовательских действий отсутствуют.

Заключение

В результате разработано компьютерное приложение «Балда» на языке программирования C++ с использованием графического фреймворка Qt. Приложение представляет собой компьютерную реализацию классической словесной игры с графическим интерфейсом пользователя.

Поставленная цель работы – разработка компьютерного приложения «Балда» для закрепления полученных теоретических знаний и практических навыков в области объектно-ориентированного программирования – достигнута. Решены все задачи, определённые к данной работе:

1. Изучены особенности предметной области, связанной с реализацией игрового приложения «Балда», и сформулированы требования к разрабатываемому программному продукту.
2. Проведен обзор языка программирования C++ и выбран инструменты разработки – qtcreator.
3. Выбран графический фреймворк и другие необходимые библиотеки для создания пользовательского интерфейса.
4. Спроектирован графический интерфейс приложения и описаны алгоритмы обработки игровых данных.
5. Разработана структура классов приложения и реализовано их взаимодействие.
6. Реализован программный код приложения с применением принципов объектно-ориентированного программирования.
7. Проведено тестирование разработанного программного продукта.

В процессе выполнения работы сформированы важные профессиональные компетенции: углублено понимание принципов работы современных информационных технологий и программных средств, получен практический опыт их использования при решении задач профессиональной деятельности, развиты навыки разработки алгоритмов и программ, пригодных для практического применения. Закреплены знания в области объектно-

ориентированного программирования, освоены современные инструменты разработки графических приложений, получен опыт проектирования программных систем и организации эффективного взаимодействия компонентов.

Разработанное приложение «Балда» готово к практическому использованию и демонстрирует навыки применения языка C++ и фреймворка Qt для создания приложений с графическим интерфейсом. Приложение имеет следующий функционал: компьютерную реализацию классической словесной игры, в которой пользователь составляет слова на игровом, добавляя по одной букве за ход и формируя слова; обеспечение управлением игровым процессом через графический интерфейс с кнопками запуска, паузы и подтверждения слов, отображением статистики составленных слов и контролированием времени на выполнение хода с помощью двухминутного таймера обратного отсчёта. Приложение обладает перспективами для дальнейшей работы, путём расширения функциональности, включая добавление сетевого режима игры, расширение словарной базы, реализацию различных уровней сложности искусственного интеллекта и сохранение статистики игр.

Список используемых источников

1. Определение классов [Электронный ресурс]: metanit. URL: <https://metanit.com/cpp/tutorial/5.1.php> (дата обращения: 10.09.2025).
2. Конструкторы (C++) [Электронный ресурс]: Microsoft Learn, 2025. URL: <https://learn.microsoft.com/ru-ru/cpp/cpp/constructors-cpp?view=msvc-170> (дата обращения: 15.09.2025).
3. Деструкторы (C++) [Электронный ресурс]: Microsoft Learn, 2025. URL: <https://learn.microsoft.com/ru-ru/cpp/cpp/destructors-cpp?view=msvc-170> (дата обращения: 15.09.2025).
4. Константы [Электронный ресурс]: metanit. URL: <https://metanit.com/cpp/tutorial/2.5.php> (дата обращения: 23.09.2025).
5. Операторы try, throw и catch (C++) [Электронный ресурс]: Microsoft Learn, 2025. URL: <https://learn.microsoft.com/ru-ru/cpp/cpp/try-throw-and-catch-statements-cpp?view=msvc-170> (дата обращения 02.10.2025).
6. Параметры функции [Электронный ресурс]: metanit. URL: <https://metanit.com/cpp/tutorial/3.2.php> (дата обращения: 09.10.2025).
7. Типы наследования: public, private и protected в C++ [Электронный ресурс]: Фоксфорд. – URL: <https://foxford.ru/wiki/informatika/tipy-nasledovaniya-public-private-i-protected-v-s> (дата обращения: 16.10.2025).
8. Наследование и полиморфизм [Электронный ресурс]: Справочник по C++. Яндекс.Учебник, 2024. URL: <https://education.yandex.ru/handbook/cpp/article/inheritance-and-polymorphism> (дата обращения: 23.10.2024).
9. Шаблоны (C++) [Электронный ресурс]: Microsoft Learn, 2025. URL: <https://learn.microsoft.com/ru-ru/cpp/cpp/templates-cpp?view=msvc-170> (дата обращения 02.11.2025).

Приложение А

(обязательное)

Код реализации приложения

На листинге А.1 представлен код реализации класса BaldaBoard.

```
#include "baldaboard.h"
#include <QGridLayout>
#include <QFile>
#include <QTextStream>
#include <QMessageBox>

BaldaBoard::BaldaBoard(QWidget *parent)
    : QFrame(parent),
      curPos(-1),
      addLetter(false),
      isStarted(false),
      isPlayerTurn(true)
{
    setFrameStyle(QFrame::Panel | QFrame::Sunken);
    setEnabled(false);

    board = new QButtonGroup(this);
    board->setExclusive(false);

    QGridLayout *layout = new QGridLayout(this);

    for (int i = 0; i < rowSize; ++i) {
        for (int j = 0; j < colSize; ++j) {
            QPushButton *btn = new QPushButton("");
            btn->setMinimumSize(50, 50);
            btn->setStyleSheet(
                "QPushButton { font-size: 18pt; font-weight: bold; }"
            );

            int id = i * colSize + j;
            board->addButton(btn, id);
            layout->addWidget(btn, i, j);
        }
    }

    connect(board, SIGNAL(buttonClicked(int)),
            this, SLOT(onBoardClicked(int)));
    connect(&timer, SIGNAL(timeout()),
            this, SLOT(timeOutSlot()));

    loadDictionary();
}

void BaldaBoard::loadDictionary()
{
    QFile file(":/dict/word_rus.txt");
    if (!file.open(QIODevice::ReadOnly | QIODevice::Text))
        return;

    QTextStream in(&file);
    while (!in.atEnd()) {
        QString word = in.readLine().trimmed().toUpper();
        if (!word.isEmpty())
            dictionary.insert(word);
    }
}
```

```

    }
}

void BaldaBoard::curletter(const QString &l)
{
    letter = l;
}

void BaldaBoard::start()
{
    isStarted = true;
    isPlayerTurn = true;
    addLetter = false;
    player.clear();
    usedWords.clear();
    curPos = -1;

    setEnabled(true);

    time = QTime(0, 2, 0);
    emit timeChanged(time.toString("mm:ss"));
    timer.start(1000);

    placeStartWord();
}

void BaldaBoard::placeStartWord()
{
    QString word = "БАЛДА";
    int row = rowSize / 2;

    for (int i = 0; i < word.size(); ++i)
        board->button(row * colSize + i)->setText(QString(word[i]));

    usedWords.insert(word);
}

bool BaldaBoard::checkWord(const QString &word)
{
    return dictionary.contains(word) && !usedWords.contains(word);
}

void BaldaBoard::apply()
{
    if (!isPlayerTurn || playerWord.isEmpty())
        return;

    if (!checkWord(playerWord)) {
        QMessageBox::warning(
            this,
            "Ошибка",
            "Слова \"" + playerWord + "\"" нет в словаре"
        );

        return;
    }

    usedWords.insert(playerWord);
    emit pl_addWord(playerWord);

    player.clear();
    playerWord.clear();
    addLetter = false;
    curPos = -1;
}

```

```

        for (auto *b : board->buttons())
            b->setStyleSheet("QPushButton { font-size: 18pt; font-weight: bold;
}");

        isPlayerTurn = false;
        computerMove();
    }

void BaldaBoard::computerMove()
{
    // ищем пустую клетку рядом с любой буквой
    for (int id = 0; id < rowSize * colSize; ++id) {
        QPushButton *cell =
            qobject_cast<QPushButton*>(board->button(id));

        if (!cell || !cell->text().isEmpty())
            continue;

        // проверяем соседей
        QVector<int> neighbors = {
            id - 1,
            id + 1,
            id - colSize,
            id + colSize
        };

        for (int n : neighbors) {
            if (n < 0 || n >= rowSize * colSize)
                continue;

            QPushButton *nb =
                qobject_cast<QPushButton*>(board->button(n));

            if (!nb || nb->text().isEmpty())
                continue;

            // пробуем все буквы алфавита
            QString alphabet = "АБВГДЕЖЗИЙКЛМНОПРСТУФХЦЧШЩЪЫЬЭЮЯ";

            for (QChar c : alphabet) {
                QString word;
                word += c;
                word += nb->text();

                if (checkWord(word)) {

                    cell->setText(QString(c));
                    usedWords.insert(word);
                    emit comp_addWord(word);

                    // передаём ход игроку
                    isPlayerTurn = true;
                    time = QTime(0, 2, 0);
                    emit timeChanged(time.toString("mm:ss"));
                    timer.start(1000);
                    return;
                }
            }
        }
    }

    // если вообще ничего не нашли — просто передаём ход
    isPlayerTurn = true;
}

```

```

        time = QTime(0, 2, 0);
        emit timeChanged(time.toString("mm:ss"));
        timer.start(1000);
    }

void BaldaBoard::onBoardClicked(int id)
{
    if (!isStarted || !isPlayerTurn)
        return;

    QPushButton *btn =
        qobject_cast<QPushButton*>(board->button(id));

    if (!btn)
        return;

    // постановка новой буквы
    if (!addLetter && btn->text().isEmpty()) {
        if (letter.isEmpty())
            return;

        btn->setText(letter);
        btn->setStyleSheet("QPushButton { color: red; font-size: 18pt; }");

        addLetter = true;
        player << btn;
        playerWord = letter;
        curPos = id;
        return;
    }

    if (btn->text().isEmpty())
        return;

    // отмена
    if (!player.isEmpty() && btn == player.last()) {
        btn->setStyleSheet("QPushButton { font-size: 18pt; font-weight: bold;
}");
        player.pop_back();
        playerWord.chop(1);
        return;
    }

    // добавление буквы
    if (!player.contains(btn)) {
        player << btn;
        playerWord += btn->text();
        btn->setStyleSheet("QPushButton { color: red; font-size: 18pt; }");
    }
}

void BaldaBoard::timeOutSlot()
{
    time = time.addSecs(-1);
    emit timeChanged(time.toString("mm:ss"));

    if (time.minute() == 0 && time.second() == 0) {
        timer.stop();
        if (isPlayerTurn)
            computerMove();
    }
}

```

```

void BaldaBoard::pause()
{
    timer.isActive() ? timer.stop() : timer.start(1000);
}

```

Листинг А.1 – Код реализации класса BaldaBoard

На листинге А.2 представлен код реализации BaldaWindow.

```

#include "baldawindow.h"
#include <QWidget>
#include <QHBoxLayout>
#include <QGridLayout>
BaldaWindow::BaldaWindow(QWidget *parent)
    : QMainWindow(parent)
{
    QWidget *central = new QWidget(this);
    setCentralWidget(central);
    board = new BaldaBoard(this);
    startBtn = new QPushButton("Start");
    pauseBtn = new QPushButton("Pause");
    applyBtn = new QPushButton("Apply");
    timeLabel = new QLabel("02:00");
    letterLabel = new QLabel("Буква: -");
    playerWords = new QListWidget;
    computerWords = new QListWidget;
    QVBoxLayout *mainLayout = new QVBoxLayout;
    QHBoxLayout *topLayout = new QHBoxLayout;
    QHBoxLayout *centerLayout = new QHBoxLayout;
    QVBoxLayout *rightLayout = new QVBoxLayout;
    topLayout->addWidget(startBtn);
    topLayout->addWidget(pauseBtn);
    topLayout->addWidget(applyBtn);
    topLayout->addStretch();
    topLayout->addWidget(new QLabel("Время:"));
    topLayout->addWidget(timeLabel);
    rightLayout->addWidget(new QLabel("Игрок"));
    rightLayout->addWidget(playerWords);
    rightLayout->addWidget(new QLabel("Компьютер"));
    rightLayout->addWidget(computerWords);
    rightLayout->addWidget(letterLabel);
    centerLayout->addWidget(board, 1);
    centerLayout->addLayout(rightLayout);
    mainLayout->addLayout(topLayout);
    mainLayout->addLayout(centerLayout);
    createLetterButtons(mainLayout);
    central->setLayout(mainLayout);
    connect(startBtn, SIGNAL(clicked()),
            this, SLOT(onStartClicked()));
    connect(pauseBtn, SIGNAL(clicked()),
            board, SLOT(pause()));
    connect(applyBtn, SIGNAL(clicked()),
            board, SLOT(apply()));
    connect(board, SIGNAL(timeChanged(QString)),
            timeLabel, SLOT(setText(QString)));
    connect(board, &BaldaBoard::pl_addWord,
            this, [this](const QString &word){
                playerWords->addItem(word);
            });
    connect(board, &BaldaBoard::comp_addWord,
            this, [this](const QString &word){
                computerWords->addItem(word);
            });
}

```

```

}
void BaldaWindow::createLetterButtons(QVBoxLayout *parentLayout)
{
    letterGroup = new QButtonGroup(this);
    letterGroup->setExclusive(true);
    QGridLayout *grid = new QGridLayout;
    QString letters = "АБВГДЕЖЗИЙКЛМНОПРСТУФХЦЧШЩЪЫЬЭЮЯ";
    int row = 0;
    int col = 0;
    for (int i = 0; i < letters.size(); ++i) {
        QPushButton *btn = new QPushButton(QString(letters[i]));
        btn->setCheckable(true);
        btn->setMinimumSize(30, 30);
        letterGroup->addButton(btn);
        grid->addWidget(btn, row, col);
        col++;
        if (col == 8) {
            col = 0;
            row++;
        }
    }
    parentLayout->addLayout(grid);
    connect(letterGroup,
            SIGNAL(buttonClicked(QAbstractButton*)),
            this,
            SLOT(onLetterClicked(QAbstractButton*)));
}
void BaldaWindow::onLetterClicked(QAbstractButton *button)
{
    board->curletter(button->text());
    letterLabel->setText("Буква: " + button->text());
}
void BaldaWindow::onStartClicked()
{
    playerWords->clear();
    computerWords->clear();
    board->start();
}

```

Листинг А.2 – Код реализации класса BaldaWindow

На листинге А.3 представлен код реализации класса BaldaWord.

```

#include "baldaword.h"

BaldaWord::BaldaWord() {}
void BaldaWord::loadDictionary(const QSet<QString>& dict)
{
    dictionary = dict;
}
bool BaldaWord::exists(const QString& word) const {
    return dictionary.contains(word);
}

bool BaldaWord::wasUsed(const QString& word) const {
    return usedWords.contains(word);
}

void BaldaWord::addUsed(const QString& word) {
    usedWords.insert(word);
}

```

Листинг А.3 – Код реализации класса BaldaWord